Relatório de Arquitetura e Engenharia de Dados: Ecossistema E.N.E.I.A.S. (ALEP)

Data: 22 de Março de 2026
Autor: Senior Software Engineer & Data Architect
Versão: V1

---

1. A Filosofia E.N.E.I.A.S. e a Arquitetura Orientada a Dados

O desenvolvimento do painel analítico da Assembleia Legislativa do Paraná (ALEP) foi rigorosamente fundamentado na arquitetura **E.N.E.I.A.S.** (Ecossistema Nativo de Extração, Investigação e Aprendizado Supervisionado). Essa filosofia prega a *Separation of Concerns* (Separação de Responsabilidades) dividindo o ecossistema em três camadas imutáveis: **Diretriz** (o escopo do negócio), **Orquestração** (o raciocínio inteligente via LLM) e **Execução** (o determinismo dos scripts em Python).

Neste contexto, um dos pilares mais críticos da nossa engenharia de dados é a Governança Efêmera estruturada através do diretório `.tmp/`. Sob a regra do E.N.E.I.A.S., o `.tmp/` atua como um *Data Lake* transitório. Todos os dados brutos (JSONs raspados das APIs governamentais) são despejados exclusivamente nesta pasta. Isso garante um isolamento perfeito entre a base de dados primária (suja, pesada e inexplorada) e o entregável final (o *Dashboard*). A pasta `.tmp/` é efêmera e pode ser expurgada e recriada a qualquer momento sem impacto no código central, assegurando conformidade com regras de *Privacy by Design* e rastreabilidade total de falhas na extração.

2. Pipeline de Ingestão: A Fase E.N.E. (Ecossistema Nativo de Extração)

Para assegurar escalabilidade e evitar a volatilidade do DOM estrutural na web, a equipe de engenharia pivotou do *Web Scraping* tradicional em HTML para a adoção massiva das **APIs Públicas Baseadas em REST**. O tráfego de matrizes JSON tipadas reduz exponencialmente a probabilidade de falhas de *parsing* (Parsing Exceptions) e sobrecarga no servidor alvo. O fluxo de ingestão foi fragmentado em três módulos executivos desacoplados:

2.1. O Motor Principal: `coletar_proposicoes_api.py`

A finalidade primária deste script é iterar de forma paginada pelo *endpoint* de filtragem da ALEP, consolidando a espinha dorsal do nosso banco de dados. Ele realiza varreduras ano a ano (2023 a 2026), ajustando os *headers* de requisição e burlando restrições HTTP através de mecanismos de *retry* exponencial.

```
# Trecho: Lógica de Paginação e Endpoint no coletar_proposicoes_api.py
url = "http://webservices.assembleia.pr.leg.br/api/public/proposicao/filtrar"
payload = {"ano": ano, "pagina": pagina, "tamanhoPagina": 100}

response = requests.post(url, json=payload, headers=HEADERS, verify=False)
dados = response.json()
if not dados.get('itens'):
   break # Fim da paginação
```

Este script é responsável por capturar o identificador mestre (`id`), a ementa, o número e o ano da proposição, salvando cada página em um *shard* JSON na pasta `.tmp/` (ex: `prop_2023_pg1.json`). Ao fim de sua execução, obteve-se um extrato de mais de **15.300 registros** únicos no *pool* efêmero.

2.2. O Enriquecimento de Dados: `coletar_detalhes_api.py`

Proposições brutas carecem de contexto situacional, como o "Local de Tramitação Atual" e as "Normas Legais" vinculadas à aprovação do projeto. Utilizando os IDs gerados no passo anterior, este programa atua como um enriquecedor (*Data Augmenter*). Para contornar a latência de milhares de disparos HTTP sequenciais, implementamos I/O Assíncrono e paralelismo via `ThreadPoolExecutor` da biblioteca nativa `concurrent.futures`.

```
# Trecho: Paralelismo na coleta de detalhes
import concurrent.futures

with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
   futures = {
       executor.submit(baixar_detalhe_proposicao, pid): pid
       for pid in proposicoes_ids_faltantes
   }
   for future in concurrent.futures.as_completed(futures):
       resultado = future.result()
```

Cada arquivo retornado da API (como `detalhe_64722.json`) salva o *trace* do processo legislativo, tornando-se a base de conhecimento para que a inteligência do sistema saiba exatamente onde um Projeto de Lei congelou ou prosperou.

2.3. O Subsistema de Validação: `coletar_normas_api.py`

Para garantir a veracidade do selo "Leis Aprovadas", este módulo acopla-se ao repositório de Normas Legais oficial da ALEP (`/api/public/norma-legal/filtrar`). Ele funciona como o validador definitivo: só considera o trabalho legislativo consumado se ele estiver carimbado na base sancionatória do Diário Oficial. Retorna os descritivos jurídicos como "Lei 21345/2023", salvando-os em `.tmp/normas_ano.json`.

3. A Fase de Síntese: Processamento e Estruturação Final

Com os dados efêmeros empilhados na pasta `.tmp/`, o *script* mais denso de nossa arquitetura entra em ação, mesclando e transmutando gigabytes de texto em um painel semântico.

3.1. O Core Analítico: `agrupar_dados_dashboard.py`

Este não é apenas um script de junção, mas sim um ETL (Extract, Transform, Load) tático. Ele varre o repositório bruto, mapeia listas vinculadas de autores em tempo (O(N)) de complexidade e escreve o artefato de *frontend* `DADOS_ALEP_20.js`. Seus principais determinismos lógicos (Regras de Negócio) compreendem:

a) O Filtro de Ruído Institucional (Data Sanitization) O JSON bruto traz "Comissões" e "Poder Executivo" como se fossem parlamentares autônomos. Atuamos com lógicas de exclusão para higienizar o censo apenas de deputados reais.

```
# Trecho: Filtro Institucional
institucionais = [
   "PODER EXECUTIVO", "TRIBUNAL DE", "COMISSÃO",
   "MESA EXECUTIVA", "MINISTÉRIO PÚBLICO", "DEFENSORIA"
]
if any(inst in autor.upper() for inst in institucionais):
   continue # Descarta proposições impessoais
```

b) A Lógica Modular de Coautoria (Many-to-Many Relationships) Um problema histórico de extração era a monopolização do mérito legislativo ao autor primário. Este algoritmo fragmenta e itera por cada autor descrito, distribuindo cópias do projeto sob a *flag* de `is_coautor`, respeitando a produtividade coletiva do plenário.

```
# Trecho: Distribuição de Autoria e Coautoria
autores_raw = item.get("autor", "Não Informado")
lista_obras = split_autores(autores_raw) # Função customizada (Regex)

for idx, nome_autor in enumerate(lista_obras):
   if nome_autor not in DICIONARIO_DEPUTADOS:
       DICIONARIO_DEPUTADOS[nome_autor] = {"proposicoes": []}

   # Flag booleana que alimenta a Tag UI do Dashboard
   is_coautor_flag = (idx > 0)

   DICIONARIO_DEPUTADOS[nome_autor]["proposicoes"].append({
       "numero": item.get('numero'),
       "is_coautor": is_coautor_flag,
       "ementa": item.get('ementa')
```

```
    })
```

4. Orquestração e Entregável

Ao final, o programa de agrupamento serializa o mega-dicionário Python para o ecossistema Web (um *Static Payload* estruturado), e a aplicação é servida no `dashboard_20_leg_V6.html`. Esta arquitetura suprime totalmente a necessidade de um *Backend Database* complexo e oneroso (como um Cluster PostgreSQL operando no lado do cliente), terceirizando o processamento para o nosso orquestrador durante o fluxo de montagem.

O resultado é uma aplicação de estado estático extremamente veloz e portável, construída por meio de extrações precisas e em total comunhão metodológica com a Filosofia E.N.E.I.A.S.